ACKNOWLEDGEMENT

I would like to express my gratitude to **Dr. Nagegowda KS,** Department of Computer Science and Engineering, PES University, for his/ her continuous guidance, assistance, and encouragement throughout the development of UE22CS441A - Capstone Project Phase – 3.

I am grateful to the project coordinators, Dr Priyanka H, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Shylaja S S, Chairperson, Department of Computer Science and

Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. Sridhar K S, Registrar, PES University for his help.

I am deeply grateful to Dr. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J,ViceChancellor, PES University for providing me with various opportunities and enlightenment every step of the way. Finally, project could not have been completed without the continual support and encouragement I have received from my family and friends.

Abstract

The goal of the "Enhancing Cloud Data Security With Blockchain" project is to protect massive volumes of data kept in the cloud from unauthorized users. Many algorithms are used to protect data security and privacy. Confidentiality, integrity, and availability (CIA) are the goals of all systems. Nevertheless, these CIA features are not offered by the current centralized cloud storage. Thus, decentralized cloud storage is used in conjunction with blockchain to improve data security and storage methods.

In order to provide strong file-level security, the first stage makes use of dynamic Advanced Encryption Standard (AES) encryption, in which distinct keys are created for every file. Blockchain technology is used in the second stage to safely store encryption keys while preserving decentralized control and data integrity. Elliptic Curve Cryptography (ECC) enhances file sharing and transmission security even more. This strategy overcomes the drawbacks of conventional cloud security techniques by combining dynamic AES encryption with blockchain key management, providing improved defense against key compromise and unauthorized access. The suggested solution is a useful tool for guaranteeing data security in contemporary cloud infrastructures since it is flexible and scalable.

Time Table Content

CHAPTER 1

INTRODUCTION

With the rapid growth of cloud computing, data security has become the most important issue for both individuals and businesses. It is true that cloud storage systems offer several advantages, such as cost savings, scalability, and usability. However, data in the cloud is vulnerable to misuse, opportunistic access, or actual data compromise due to flaws in the stilldominant methods of encryption and centralized key management. Cloud systems that store and distribute vast amounts of sensitive data among numerous users are especially vulnerable to these security risks. This project offers a fresh approach to cloud data security by combining blockchain-based decentralized key management with dynamic AES encryption.

Every file has a unique key that is always changing thanks to dynamic AES encryption. improving security and reducing the impact of key compromise. Additionally, the use of blockchain technology increases the security and integrity of the data by providing a decentralized, unchangeable method of storing these encryption keys. Curve Elliptical The goal of the suggested solution is to solve the issues that the current encryption methods are having, which is why it is important to have a more flexible and reliable security architecture that can handle the difficulties presented by cloud computing threats. The project provides a complete solution for improving the confidentiality, integrity, and security of data stored in the cloud by utilizing these cutting-edge technologies.

CHAPTER 2

PROBLEM DEFINITION

Cloud-based data processing and storage is now commonplace. Maintaining data integrity and using safe storage methods are still difficult, though. Despite being widely used, encryption techniques are typically insufficient due to numerous flaws. Centralized key management systems in particular are extremely vulnerable since a single compromised key can grant access to a large amount of private information. Additionally, this concentration creates a single point of failure that increases the system's risk of data leaks.

Given these difficulties, it makes sense to highlight the need for a more reliable, decentralized, and adaptable encryption key management system to safeguard user data stored in the cloud. Any proposed system must ensure that unauthorized access to data is never permitted and that the extent of damage in the event that a key is compromised is minimal. Additionally, the solution should offer a particularly scalable and adaptable model that aims to enhance data security in the various clouds and enable safe user communication. This study aims to improve cloud data security by combining blockchainbased decentralized key management with dynamic AES encryption.

CHAPTER 3

LITERATURE SURVEY

3.1 Reserch Paper 1

S. Rehman, N. Talat Bajwa, M. A. Shah, A. O. Aseeri, and A. Anjum,"Hybrid AES-ECC model for the security of data over cloud storage,"Electronics, vol. 10, no. 21, p. 2673, Oct. 2021

In order to improve data security, the researchers in this study proposed a hybrid model for cloud storage that combines Elliptic Curve Cryptography (ECC) and the Advanced Encryption Standard (AES). Data was encrypted using the AES algorithm, and secure keys were generated using ECC. The hybrid approach finds a middle ground between strong encryption and computational overhead, making it suitable for advanced cloud application security. Within the parameters of the study, the model performed faster and more securely than traditional encryption techniques, particularly in settings with limited resources like mobile cloud storage.

3.2 Reserch Paper 2

S. K. Dwivedi, R. Amin, J. D. Lazarus, and V. Pandi, "Blockchain based electronic medical records system with smart contract and consensus algorithm in cloud environment," Secur. Commun. Netw., vol. 2022, pp. 1–10, Sep. 2022.

The goal of this study was to improve EMRs by integrating blockchain technology into the cloud. The study created a technique to use a distributed blockchain ledger to store medical

data's encryption keys and metadata in an unchangeable manner. To preserve the integrity of the data defense against unauthorized use, the blockchain's mechanical integration was used.

The Improved security was achieved through research using medical cloud systems without sacrificing data privacy or health professionals' access to the data.

3.3 Reserch Paper 3

G. Habib, S. Sharma, S. Ibrahim, I. Ahmad, S. Qureshi, and M. Ishfaq, "Blockchain technology: Benefits, challenges, applications, and integration of blockchain technology with cloud computing," Future Internet, vol. 14, no. 11, p. 341, Nov. 2022.

To help with data security in cloud environments, they proposed a Fine-Grained Access Control (FGAC) system that included fuzzy logic. While biometric physical characteristics were used to verify access, the system generated public, private, and session keys to encrypt data. By utilizing several security layers, including multi-layer encryption, key distribution management, and biometric identification, the solution was specifically designed to prevent unauthorized access to personal data. In this manner, strong protection against cyberattacks was guaranteed.

3.4 Reserch Paper 4

K. Bhalla, D. Koundal, S. Bhatia, M. Khalid Imam Rahmani, and M. Tahir,

"Dynamic encryption and secure transmission of terminal data files," Comput., Mater. Continua, vol. 71, no. 1, pp. 1221–1232, 2022

This inquiry revealed something new. In 2017, Soliman B 166 Aronsohn & Barber 166 reported This should be removed since it is repetitive. NLCA is a lightweight cryptographic algorithm with unique features that modifies the number of blocks but not the number of block ciphers.

The entire key is sixteen bytes long because the XOR is still sixteen bytes long. Because speed and resource usage are crucial in cloud services, the algorithm was able to encrypt at faster speeds while maintaining security. To bolster this claim, a comparative analysis with existing algorithms NLCA effectively aided security without wasting time, as demonstrated by Reiss et al. 167.

3.5 Reserch Paper 5

I. Keshta, Y. Aoudni, M. Sandhu, A. Singh, P. A.Xalikovich, A. Rizwan, M. Soni, and S. Lalar, "Blockchain aware proxy re-encryption algorithm based data sharing scheme," Phys. Commun., vol. 58, Jun. 2023,

Art. no. 102048. To solve the problems of confidentiality and privacy of the information stored in the clouds, the researchers implemented a Non-Deterministic Cryptographic Scheme (NCS).

TheAllon completed with the aid of the Sliding Window Algorithm (SWA) who also utilizes the Linear Congruential Generator (LGC), thus, making it extremely robust against interference through data analysis techniques. The scheme performed better than the most accepted algorithms like AES, RSA, and DES in encryption time execution and strength.TheNCS efficiently encodes problems and is mobile within the confines of clouddatasecurity.

CHAPTER 4

Project Requirements Specification Document

4.1 Title

Enhancing Cloud Data Security with Blockchain

4.2 Purpose

The goal of this project is to overcome the drawbacks of conventional centralized storage by designing and implementing a secure cloud storage system. The project intends to guarantee Confidentiality, Integrity, and Availability (CIA) of data stored in the cloud by combining.

Elliptic Curve Cryptography (ECC), Blockchain-based key management, and Dynamic AES Encryption.

4. 3 Scope

The project's main goal is to improve data security in cloud environments during storage, transmission, and access.

It will offer:

• Use dynamic AES for secure file encryption with distinct keys for each file.

Blockchainoriented

• Decentralized key storage to avoid unauthorized key access and single-point failure Secure communication between users using ECC.

• For businesses or individuals who need to safely store and share sensitive data on the cloud, the suggested solution is scalable, modular, and appropriate.

4.4 Objectives

1. To improve cloud data storage's availability, confidentiality, and integrity.
2. To use dynamic AES encryption to protect data at the file level.

3. To incorporate blockchain technology for decentralized management of encryptionkeys.
4. To use Elliptic Curve Cryptography (ECC) for safe data sharing and transmission.
5. To assess performance in terms of attack resistance, scalability, and security.

4.5 Functional Requirements

ID Requirement Description Priority

FR1 Users will be able to upload and safely store files in the cloud. High

FR2 A distinct AES key that is generated dynamically will be used to encrypt each file.
Elevated

FR3 Blockchain smart contracts will be used to store and manage the encryption keys.
Elevated

FR4 Authorized users will be able to decrypt files using the system by safely retrieving keys from the blockchain technology. For safe file sharing between users, high

FR5 ECC must be utilized. Medium

FR6 All access and modification activities must be recorded by the system for auditing purposes.

FR7 Before performing any file operations, the system must guarantee user authentication and access control (Medium). High

FR8 In the event of a partial system failure, the system must offer recovery mechanisms.
Moderate

4.6 Non-Functional Requirements

1. Safety must guarantee tamper-proof key storage and end-to-end encryption.
2. Performance: Encryption and decryption should be quick and effective.
3. Dependability Data integrity must be maintained because blockchain guarantees that there is no single point of failure.
4. Scalability Large datasets and numerous concurrent users should be supported.
5. Usability Interface should be simple for file upload, download, and sharing.

Maintainability Code should be modular for easy updates and integration with other systems.

4.7 System Requirements

4.7.1 Hardware Requirements

• Processor: Intel Core i5 or above

• RAM: Minimum 8 GB

• Storage: Minimum 500 GB HDD / 256 GB SSD

• Internet Connection: Required for blockchain node interaction

4.7.2 Software Requirements

• Operating System: Windows / Linux

• Programming Language: Python

• Frameworks: Django

• Blockchain: Ethereum / Hyperledger Fabric (for decentralized key management)

• Database: SQ ite3

IDE: Visual Studio Code

4.8 System Model

4.8.1 Architectural Design

The system consists of three main components:

1. User Interface Layer: Used for file management, uploading, and downloading.
2. Encryption & Transmission Layer: Manages secure sharing based on ECC and AESencryption/decryption
3. Blockchain Layer: Uses smart contracts to control access and store encryption keys.

4.8.2 Data Flow Summary

1. User uploads file → AES key generated → File encrypted →
2. AES key stored in blockchain → Encrypted file stored in cloud → Authorized user 3.
3. retrieves key → File decrypted.

4.8.3 Constraints

1. High computational costs for blockchain transactions and key generation.
2. Blockchain interaction requires internet access.
3. The decentralized setup may result in higher initial deployment costs.

4.9 Future Enhancements

• Integration for decentralized file storagewithIPFS(InterPlanetaryFileSystem).
• Setting up support for multiple clouds.
• Using machine learning to identify unusualintrusionsoraccesspatterns

4.10 Expected Outcome

1. A working prototype that uses Blockchain, ECC, and Dynamic AES to demonstrate safe file sharing and storage.
2. Improved defense against illegal access and key compromise.
3. Enhanced scalability, transparency, and trust in cloud-based data systems.

CHAPTER 5

High-Level Design (HLD)

5.1 Introduction

5.1.1 Purpose of this Document

The architecture, components, data flows, interfaces, non-functional requirements, and deployment considerations for the project "Enhancing Cloud Data Security With Blockchain" are all described in this High-Level Design (HLD). The goal of the HLD is to close the gap between the low-level design (LLD) and implementation and the system requirements.

5.1.2 Scope

The HLD covers the design of a cloud-based file storage solution that uses ECC for secure transmission and sharing, blockchain-based decentralized key management (smart contracts store key references/hashes, access policies, and audit trails), and dynamic AES encryption (perfile unique and periodically rotated keys). Web and mobile client components, server-side services, blockchain network components (public or permissioned based on deployment), and integration with cloud object storage are all included in this.

5.1.3 Definitions, Acronyms and Abbreviations

AES: Advanced Encryption Standard

ECC: Elliptic Curve Cryptography

CIA: Confidentiality, Integrity, Availability

KMS: Key Management System

API: Application Programming Interface

DB: Database

S3: Generic object storage (used as example)

Smart Contract: Blockchain program storing references, metadata and access logs

5.2 System Overview

5.2.1 Goals and Design Principles

1. File-level confidentiality: Every file is encrypted using a different AES key.
2. Reduce blast radius: Ephemeral and dynamic keys lessen compromise-related damage.
3. Decentralized key control: To prevent a single point of failure, store key metadata and access control on a blockchain.
4. Secure transmission and sharing: Establish sessions, use digital signatures, and exchange secure keys using ECC.
5. Cloud-native and scalable: When feasible, make components stateless and scale services horizontally.
6. Auditability: Unchangeable record of key issuance, rotation, and access through

5.2.2 High-level Architecture Diagram (textual)

1. Users (Web/Mobile) ⇄ API Gateway ⇄ {Authentication Service}
2. API Gateway → Encryption Service (Dynamic AES) → Cloud Storage (Object store)
3. Encryption Service ↔ Blockchain Network (Smart Contract) ↔ Key metadata & audit
4. API Gateway ↔ ECC Module (for secure sessions / signatures)
5. Monitoring & Logging, Admin Console, CI/CD

5.3. Components and Subsystems

5.3.1 Client / User Interface

1. Files can be uploaded, downloaded, and shared via a web or mobile application.
2. When necessary, establishes secure sessions by performing client-side ECC-based key exchange.
3. For end-to-end encryption scenarios, client-side encryption is optional (design choice).

5.3.2 API Gateway

1. Authentication/authorization enforcement (JWT/OAuth2).
2. Routing to microservices, rate limitation, and request validation.

5.3.3 Encryption Service (Dynamic AES)

1. Uses a secure random generator (CSPRNG) and, while processing, keeps temporary keys in secure memory.
2. EncryptFile, DecryptFile, RotateKey, and RekeyFile are among the functions it offers.
3. Encrypted blobs are stored in cloud object storage, and the file identifier and key-reference ID are returned for blockchain recording.

5.3.4 Key Management via Blockchain

1. Access control policies, unchangeable audit logs (timestamps, user actions), and key references (such as key IDs, key metadata, hash of encrypted key or key-encryption-key ciphertext) are all stored in smart contracts.
2. The actual AES key material shouldn't be transparently stored on the blockchain; instead, it is used to record references.
3. Rather, we use a keyencryption system to store encrypted key material.

5.3.5 ECC-based Secure Transmission Module

1. AES keys can be safely delivered to intended recipients using ECC-based key exchange (such as ECDH).
2. To sign transactions and guarantee non-repudiation, ECC uses digital signatures (such as ECDSA).

5.3.6 Storage Layer (Cloud Storage)

1. Blobs of encrypted files stored in object storage (compatible with S3).
2. File id, owner, pointer to blockchain key-id, encryption algorithm version, size, and timestamps are examples of metadata kept in a conventional database (Postgres/MySQL) for speedy lookup.

5.3.7 Auditing & Logging Service

Receives events from services and logs to blockchain for unchangeable records when needed, as well as centralized logging (ELK/Prometheus/Grafana).

5.3.8 Admin & Monitoring

Use the admin console to view the audit trail, manage policies, and start rotation and recovery. Keeping an eye out for security and performance issues.

5.4. Module Description

There are multiple logical modules that make up the system. Each module has a distinct set of functions and communicates with other modules via clearly defined interfaces. Scalability, maintainability, and clarity in the system's high-level design are guaranteed by this modular structure.

5.4.1 Module for User Authentication

1. Accountability: Manages user login and registration.
2. The blockchain smart contract is used to verify user credentials and activation states.
3. Authentication for cloud administrators is managed independently.

Interfaces

1. AuthService: sign up, log in, and activate users
2. Management of sessions
3. Validation of input

5.4.2 Module for Encryption

1. It is responsible for creating cryptographic keys and encrypting and decrypting files.
2. Supports ECC/Fernet for alternative secure encryption and AES-256-GCM for symmetric encryption in the prototype.

Interfaces:

EncryptionService: wrap_key(), unwrap_key(), rotate_key(), encrypt(), decrypt()

5.4.3 Module for File Upload and Storage

Accountability: Uploads encrypted file blobs to cloud storage (local static directory/Firestore) and keeps track of the relevant metadata.

Interfaces:

StorageService: get_blob(), store_blob(), get_presigned_url()

5.4.4 Module for File Sharing and Requests

1. Responsibility: Allows users to ask for access to other people's files.
2. Oversees requests that have been accepted, denied, and pending.
3. Initiates the key-sharing process (in the prototype, email delivery).

Interfaces

ShareService: request creation, approval, rejection

5.4.5 Module for File Decryption and Retrieval

1. Retrieving encrypted content and matching keys (raw or wrapped) is the responsibility.
2. Decrypts data and only gives authorized users plaintext output.

Interfaces

DecryptionService: decrypt_stream(), verify_key()

5.4.6 Module for Blockchain Interaction

1. Accountability: Communicates with the smart contract to verify users, activate them, and record key metadata in the future.
2. Guarantees authorization logic's immutability and reliability.

Interfaces

BlockchainService: loginFunction(), AddUsers(), getUsersActivated(), updateState(), registerFile() (future), grantAccess() (future).

5.4.7 Database Module for Firestore

Collections

Encrypted files

Filerequest

Encrypted file metadata, file-request data, timestamps, algorithms, keys (prototype), and request lifecycle status are all stored.

5.4.8 Module for Email Notifications

Accountability: After the owner grants access, decryption keys are sent to authorized users.

Uses SMTP for communication.

(Secure key-exchange via ECC or KMS replaces this in production systems.)

5.4.9 Module for Cloud Administration

Accountability: Enables cloud administrators to oversee high-level operations, monitor system activity, and activate or deactivate users.

5.5. Data Flow and Sequence Diagrams

5.5.1 Upload Flow (summary)

1. The user uses the Auth Service to authenticate (gets JWT).
2. The client asks the API Gateway for an upload session.
3. To create the AES file_key, the API contacts the Encryption Service.
4. The encryption service encrypts the file, saves it in Object Storage, and encapsulates the file_key with the Key Encryption Key (KEK) or recipient public key.
5. The encryption service writes the encrypted-wrapped-key hash and key reference to the blockchain's smart contract.
6. The user receives the file-id and transaction-id from the API.

5.5.2 Download / Share Flow (summary)

1. User requests file download.
2. API validates authorization (via blockchain-stored policies and DB metadata).
3. If permitted, the API asks the smart contract or off-chain store for wrapped file_key.
4. To retrieve the AES key, KEK unwrap or ECC-based key exchange takes place.
5. The user downloads the file after the encryption service decrypts it (or sends the encrypted blob back to the client for client-side decryption).

5.5.3 Key Rotation Flow

1. Rotation is requested by the administrator or an automated scheduler.
2. The encryption service creates a new AES key, re-encrypts the file or, if the keywrapping method is being used, re-wraps the key without completely re-encrypting it, and updates the smart contract with the new key reference and audit entry.

5.6. Detailed Component Design

5.6.1 Encryption Service (AES) — internals

AES mode: AES-256-CBC + HMAC if GCM is not available, or AES-256-GCM (recommended for confidentiality + integrity).

Important lifecycle:

1. Generation: To create a 256-bit key for each file, use CSPRNG.
2. Store the IV along with the ciphertext and use a unique IV/nonce for each encryption.
3. Tag is kept with the ciphertext (for GCM).

Formats for storage:

- Key wrapping: wrapped_key is created by encrypting file_key using KEK or recipient public key (ECC).
- Stored as an encrypted blob or off-chain, with its hash being recorded on-chain to ensure integrity.
- Format for an encrypted blob: {version, algorithm, iv, ciphertext, auth_tag}

5.6.2 Blockchain Key Store — internals

Functions of smart contracts:

registerFile (owner, policy, metadataHash, wrappedKeyHash,
fileId) updateKey (timestamp, newWrappedKeyHash, fileId)
grantAccess (conditions, grantee, fileId) logAccessEvent (action,
timestamp, fileId, and user)
On-chain data structures: fileId → list of ACL entries + CM (content
metadata) + keyhash entries hash
Off-chain components include encrypted wrapped keys kept in a secure database or object store, a secure key vault or HSM for KEKs, and only hashes and pointers stored on the blockchain.

5.6.3 ECC Module — internals

Curves: secp256r1 (P-256) or Curve25519 (X25519) for ECDH.
Use ECC for signing important operations (upload registration, key rotation) and for establishing session keys.

5.7 REST API Endpoints (HLD – simplified)

5.7.1 POST /files/upload-session

Purpose: Start file upload, generate AES key server-side.
Returns: file_id, upload path, wrapped-key hash (for blockchain).
Prototype equivalent: /encryptdata

5.7.2 POST /files/{fileId}/complete

Purpose: Finalize upload and register metadata + key-hash on blockchain.
Prototype: part of /encryptdata flow.

5.7.3 GET /files/{fileId}

Purpose: Get file metadata or download link.

Prototype: /viewfiles, /viewcloudfiles

5.7.4 POST /files/{fileId}/request-access

Purpose: Request access to another user's encrypted file.

Prototype: /sendrequest/<id>

5.7.5 POST /requests/{requestId}/approve

Purpose: Owner approves request; key is sent to requester (email in prototype).

Prototype: /sendkey/<fileid>

5.7.6 POST /files/{fileId}/decrypt

Purpose: Submit key and decrypt the file.

Prototype: /viewmyfiles/<id>

5.7.7 POST /files/{fileId}/rotate-key (optional)

Purpose: Rotate encryption key.

(In prototype: not implemented but supported in design.)

- Smart Contract Interfaces (short)
- These functions correspond to your actual blockchain contract:
- checkEmail(email)
- AddUsers(name,email,password,contact,address,status)
- loginFunction(email,password)
- getUsersActivated(state)

- updateState(id,state) (your code uses upldateState)
- registerFile(fileId, owner, metadataHash, wrappedKeyHash)
- grantAccess(fileId, grantee)
- revokeAccess(fileId, grantee)
- Note: Only hashes/pointers go on chain — never the actual AES key.
- Internal Interfaces (very short)
- Firestore:
- encrypted_files (metadata, encrypted content, algorithm, paths)
- filerequest (pending/approved access requests)
- Email Service: sends AES/ECC key to approved requester (prototype)..

CHAPTER 6

Low-Level Design

6.1 Introduction

The Low-Level Design (LLD) provides a detailed technical specification of how each module in the system is implemented. It includes class-level details, data structures, internal workflows, database schemas, encryption logic, API contracts, and sequence flows that map directly to the source code of the project. This LLD bridges the gap between the HLD (conceptual architecture) and the actual implementation written in Django, Firestore, and Web3.py.

6.2 System Architecture (LLD Perspective)

From the standpoint of LLD, the architecture is made up of:

- UI logic, file encryption, database access, authentication, and email services are all handled by a Django-based application backend.
- Access requests and encrypted file metadata are stored in the Firestore database.
- Unchangeable user data is stored in a blockchain smart contract.
- Cryptographic modules that carry out ECC and AES functions.
- Encrypted file blobs are stored in a static storage layer.

The following sections provide a detailed description of the well-defined interfaces through which each subsystem interacts.

6.3 Class and Component Design

6.3.1 Obligations

Using the smart contract, users can register and authenticate themselves.

In Django, control session variables.

Internal Functions

Function	Description
Register(name, email, password, contact, address)	invokes the blockchain's AddUsers() function.
login(password, email)	retrieves user data by calling the blockchain's loginFunction().

Information Utilized

- ABI smart contract
- One account on the blockchain (ONE_ACCOUNT)
- Cookies for Django sessions

Exceptions

- Failure of a smart contract call
- Incorrect credentials

6.3.2 Encryption Service (AES + ECC)

Duties

- Create robust 32-byte AES keys.
-

Text content can be encrypted and decrypted.

- As a prototype, support ECC/Fernet encryption.

AES Internals key = 32-byte CSPRNG value iv

= 12-byte IV tag = 16-byte GCM tag ciphertext

= AES-GCM(key, iv).encrypt(plaintext)

Data Structures

Field **Description** filekey

Base64-encoded AES key textcontent

Base64({iv + tag + ciphertext}) algorithm

"aes" or "ecc"

Interfaces

- encrypt(key, plaintext)
- decrypt(enc_blob, key)

6.3.3 StorageService (Encrypted File Storage)

Duties

- Write the encrypted file to **static/AES/encryptedfiles/**.
-
-

Save the decrypted preview file in **static/AES/files/**.

Handle ECC file storage in a similar manner.

NOTE:

In production use S3/minio, here static folder is used per project requirement.

6.3.4 FileSharingService (Submit and Accept Requests)

Obligations

- Make a Firestore entry for a file access request.
- When a request comes in, let the owner know.
- Permit the owner to send the key and approve the request.

Filerequest Firestore Document Structure

Field **Description** fileid

encrypted file ID user email

requester owner email

file owner recipient email file

receiver

status pending/approved filekey

ECC key (prototype) or AES key

-
-

Interfaces create_request(requester,

file_id)

approve_request(request_id)

-
-

- send_key_via_email(key, recipient)

6.3.5 FileDecryptionService

Obligations

- Obtain the user's authorized file requests.
- Verify the decryption key.
- Decrypt data using AES or ECC and display the plaintext.

Internal Procedures

1. Get the encrypted_files document by ID.
2. Decode the ciphertext and base64 key.
3. If AES: plaintext = AES.decrypt(iv + tag + ciphertext, key)
4. Put the plaintext back into the template.

6.3.6 BlockchainService (Smart Contract Usage)

Ganache has implemented ABI functions for **UserContract.json**.

Obligations

- Create a user account → AddUsers()
- Verify login → loginFunction()
- Turn on user → upldateState()
- Obtain inactive users → getUsersActivated()

Internal Operations

Function	Description
login(password, email)	Returns (id, name, email, status, etc.)
checkEmail(email)	guarantees a distinct registration AddUsers() New user is inserted
upldateState(id, state)	The user is activated

6.3.7 FirestoreDatabaseService

Collections

- encrypted_files
- Filerequest

Encrypted Files Data Model

Field	Description useremail
Owner textcontent	Base64 encrypted
text filekey	Base64 AES/ECC
key encfilepath	Path of encrypted
file **Field**	**Description**

decfilepath Path of decrypted file

algorithm aes/ecc datetime

Timestamp

Filerequest Data Model

Field **Description** fileid

reference to encrypted file owner

file owner recipient

requester useremail

requester status

pending/approved filekey

encryption key datetime

timestamp

6.3.8 Email Service

Obligations

After approval, the requester receives the AES/ECC key.

Function send_mail("File Key", <body>, from, [to])

Note:

Instead of using plain email in production, use secure key-exchange.

6.4 Comprehensive Sequence Flows

6.4.1 LLD File Upload Flow

1. The user chooses AES/ECC after entering data.
2. A POST request is received by View.
3. Plaintext is encrypted and a key is generated by the encryption service.
4. Encrypted blobs are written to static paths by StorageService.
5. Encrypted files now have a Firestore entry.
6. A success message was sent back.

6.4.2 LLD File Request Flow

1. "Send Request" is clicked by the requester.
2. Filerequest now has a Firestore entry.
3. Pending requests are visible to the owner.
4. Owner gives his approval and sends the mail.
5. The request has been updated to "approved."

6.4.3 LLD File Decryption Flow

1. The approved items are viewed by the requester.
2. Submits the key.
3. The encrypted file is retrieved by the system.

4. Decrypts with ECC or AES.
5. The plaintext is displayed.

6.5 Design at the API Level

The views serve as endpoints even though your system does not use the Django REST API. The LLD API equivalents are as follows:

6.5.1 /login

- **Method:** POST
- **Goal:** blockchain login
- **Logic:** verify status, store session, call loginFunction.

6.5.2 /encryptdata

- **Method:** POST
- **Goal:** Encrypt the file
- **Logic:** AES/ECC → Firestore insert → save files

6.5.3 /sendrequest/

- **Method:** GET
- **Goal:** Make a file request
- **Logic:** Send a request to Firestore.

6.5.4 /sendkey/

- **Method:** GET
- **Goal:** The owner grants the request
- **Logic:** Give the requester the key.

6.5.5 /viewmyfiles/

- **Method:** POST
- **Goal:** use a key to decrypt
- **Logic:** Decrypt and display using AES/ECC logic.

6.6 Handling Errors and Edge Cases

Area	Management
Invalid Key	Display "Invalid key!"
File Not Found	Show missing document message
Blockchain Failure	use try/except on smart contract calls
Encryption Failure	Verify algorithm
Missing Session	Redirect to login

6.7 Security Points to Remember

- 32 bytes are required for the AES key (CSPRNG).
- For GCM, IV needs to be 12 bytes.
- Never use Firestore to store raw plaintext.
- Email usage is merely a prototype.
- Blockchain has no keys; it only stores metadata.

- When deploying, use HTTPS.

6.8 Database Schema Summary (LLD)

Encrypted File Table

- useremail (String)
- Base64 encrypted content
- Base64 key
- type of encryption (String)
- paths (String)
- timestamp (String)

File Requests Table

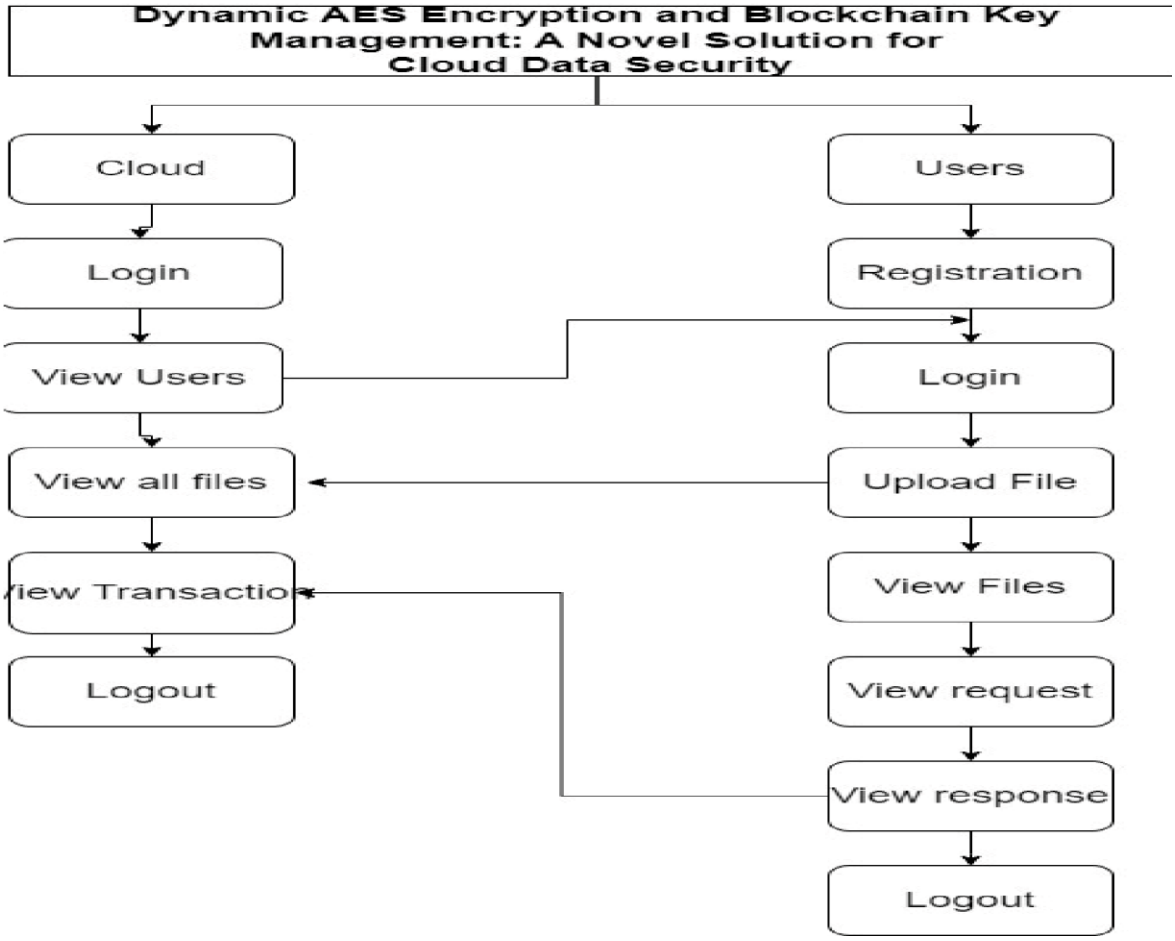
- file_id
- proprietor
- receiver
- status
- key
- timestamp

6.9 LLD Synopsis

The internal logic, data structures, algorithms, encryption techniques, and module interactions of the system **"Enhancing Cloud Data Security with Blockchain"** are described in detail in this chapter. The implementation written with Django, Firestore, AES encryption modules, email interfacing, and blockchain is directly mapped to the LLD.

CHAPTER 7

System Design



7.1 User Module Design

- **Registration:**
  In order to register, users must provide personal information like their name, email address, password, phone number, and address. For the purpose of authentication, this data is safely kept in the database.

- **Login:**
  To access the system and carry out tasks like encryption, uploading, and key management, users authenticate with their registered credentials.

- **Encrypt Data :**
  Before uploading user data to the cloud, the system encrypts it using the Advanced Encryption Standard. Strong symmetric encryption offered by AES guarantees that the information is kept private and shielded from unwanted access.

- **Upload File:**
  The file is uploaded to the cloud server after encryption is finished. For traceability, every file upload creates a matching blockchain transaction entry.

- **My Files / View Files:**
  Every file that a user has uploaded is viewable. Until a working decryption key is found, these files stay encrypted.

- **Key Request :**
  The system uses Elliptic Curve Cryptography for safe key exchange when a user asks to decrypt a file. ECC guarantees that the encryption strength stays extremely high even with small key sizes.

- **Decrypt Data:**
  The user can decrypt and access the original file content after being approved and receiving the decryption key via blockchain validation.

- **Logout:**
  To avoid unwanted access, users safely log out after finishing their tasks.

7.2 Cloud Module Design

- **Login:**
  Using the system's default credentials, the cloud administrator logs in.

- **View Users:**
  The administrator has the ability to see every registered user and keep an eye on their system activity.

- **View All Files:**
  All encrypted files kept on the cloud server can be viewed by the administrator.

- **View Transactions:**
  View Transactions: All user-to-user transactions, including file uploads, key requests, and decryptions, are shown by the cloud module. To guarantee transparency and immutability, every transaction is documented on the blockchain.

- **Logout:**
  After overseeing and managing system operations, the administrator can safely log out.

7.3 Blockchain and Security Design

- **Blockchain Integration (Ganache):**
  To handle key exchange and transaction verification, the system incorporates Ganache, a local Ethereum blockchain simulator. Transparency and unchangeable data management are ensured by recording each file encryption, key request, and decryption procedure as a transaction on the blockchain.

- **AES for Data Encryption:**
  AES protects confidentiality by dynamically encrypting user data before uploading it to the cloud.

- **ECC for Key Management:**
  ECC offers strong security with less processing power and is used for lightweight, secure key generation and exchange.

7.4 Overall Design Flows

1. After registering, the user logs in.
2. Data is uploaded by the user and encrypted with AES.
3. Cloud storage is used to store encrypted files.
4. Blockchain is used for key management and transactions (Ganache).
5. Secure key transfer for decryption is guaranteed by ECC. 6. Immutablerecords of user and cloud activity are kept.

CHAPTER 8

Methodology

The methodology outlines the methodical approach used to design and implement a secure cloud data storage system utilizing blockchain technology, Elliptic Curve Cryptography (ECC), and Dynamic AES Encryption (Ganache). Between users and the cloud, the procedure guarantees data confidentiality, integrity, and secure key management.

8.1. Methodological Steps

Step1:UserRegistrationandAuthentication

To register, new users must enter their name, email address, password, phone number, and address

Users are authenticated using their registered credentials by the login module.

Users can access file management and encryption features after authenticating.

8.2: Data Encryption Using AES

The system uses Advanced Encryption Standard (AES) encryption prior to uploading any data. AES ensures confidentiality by using a dynamic secret key to transform the original data into an unintelligible ciphertext. After that, the encrypted file is prepared for uploading to the cloud.

8.3: File Upload to Cloud

- The cloud server receives encrypted files.

A blockchain transaction is automatically created during the upload process to record the

8.4: Blockchain Key Management with Ganache

Ganache, a private Ethereum blockchain environment, is used to record all user transactions such as file uploads, key requests, and decryption approvals. Every transaction is given a distinct hash value, guaranteeing that it cannot be changed or removed.

8.5: Key Exchange Using ECC

Elliptic Curve Cryptography (ECC) is used by the system for secure key generation and transfer when a user requests a key to decrypt a file.ECC reduces computation time while maintaining strong encryption and offers high security with shorter key lengths.

8.6: Data Decryption

The user can decrypt the file after obtaining the key through blockchain verification.

To recover the original data, AES decryption is used.

8.7: Cloud Monitoring

- The default login credentials are used by the cloud administrator.
- All registered users, encrypted files, and transaction history stored on the blockchain are visible to the administrator.
- This guarantees complete auditability and transparency of system operations.

CHAPTER 9

Implementation and Pseudocode

Django was used as the backend framework, and HTML, CSS, and JavaScript were used for the frontend interface of the suggested system. User information, encrypted files, and file request data are stored in the SQLite3 database. For safe data encryption and decryption, the system combines the AES (Advanced Encryption Standard) and ECC (Elliptic Curve Cryptography) algorithms. Furthermore, Web3.py and Ganache are utilized to

oversee user registration and verification using a local Ethereum blockchain smart contract, guaranteeing data immutability and integrity.

The system gives users the option to select between AES and ECC when they upload data for encryption.

- A secret key that is generated at random and safely kept in the database is used to encrypt the data.
- For later access, the encrypted data is stored in the "static" directory.
- When a different user requests access, the cloud server uses Django's mail functionality to verify and send the decryption key to the requester's email address.
- To successfully decrypt the file, the user enters the received key.

Using secure cryptographic algorithms and blockchain verification, the entire process guarantees confidentiality, integrity, and access control. and secure cryptographic algorithms.

BEGIN

DISPLAY login page

IF user selects "Cloud Server" THEN
    VERIFY email and password with admin credentials

REDIRECT to cloud dashboard

ELSE IF user selects "User" THEN

CALL blockchain contract to verify credentials

IF user is active THEN

REDIRECT to user homepage

ELSE

DISPLAY "Account not verified"

END IF

END IF

ON registration:

GET user details (name, email, password, contact, address)

IF email does not exist on blockchain THEN
ADD user to blockchain with status = "Deactivated"

ELSE

DISPLAY "Email already exists"

END IF

ONencryption request:

GET message and selected algorithm (AES or ECC)

GENERATE random secret key

IFalgorithm = AES THEN

ENCRYPT message using AES with secret key

ELSE IF algorithm = ECC THEN

ENCRYPT message using ECC with secret key

END IF

STORE encrypted file path and key in database

DISPLAY "File Encrypted Successfully"

ONfile request:

USER selects an encrypted file

CREATE a new record in Filerequest table with status = "Pending" ON cloud approval:

CLOUD SERVER retrieves file request

SEND decryption key to receiver’s email

UPDATE request status to "Approved"

ONdecryption:

USER enters key received via email

VALIDATE key

IFkey is correct THEN

DECRYPT file and display original data

ELSE

DISPLAY "Invalid Key"

END IF

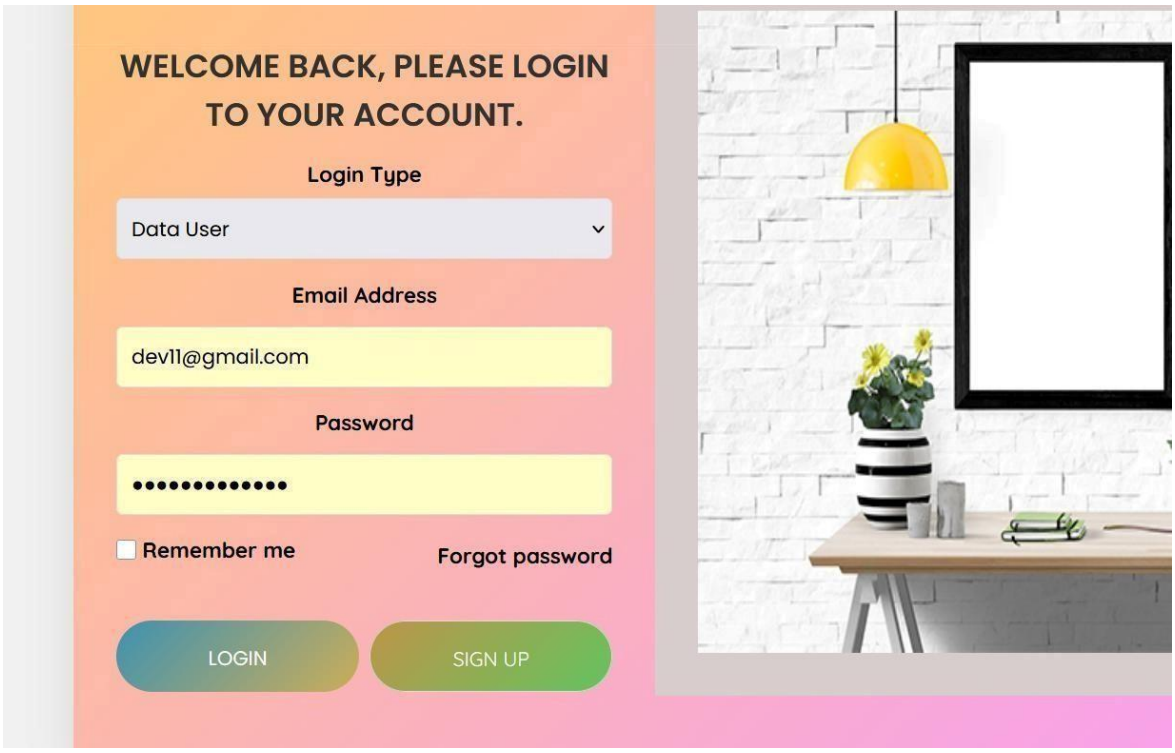
END

CHAPTER 10

Experimentation Results and Discussion

10.1 User Login Interface

Users can select their login type (Data User or Data Owner) and enter their credentials on the system's secure login page. Using SQLite3 database validation and Django's authentication mechanism, this interface guarantees authenticated access to the system.

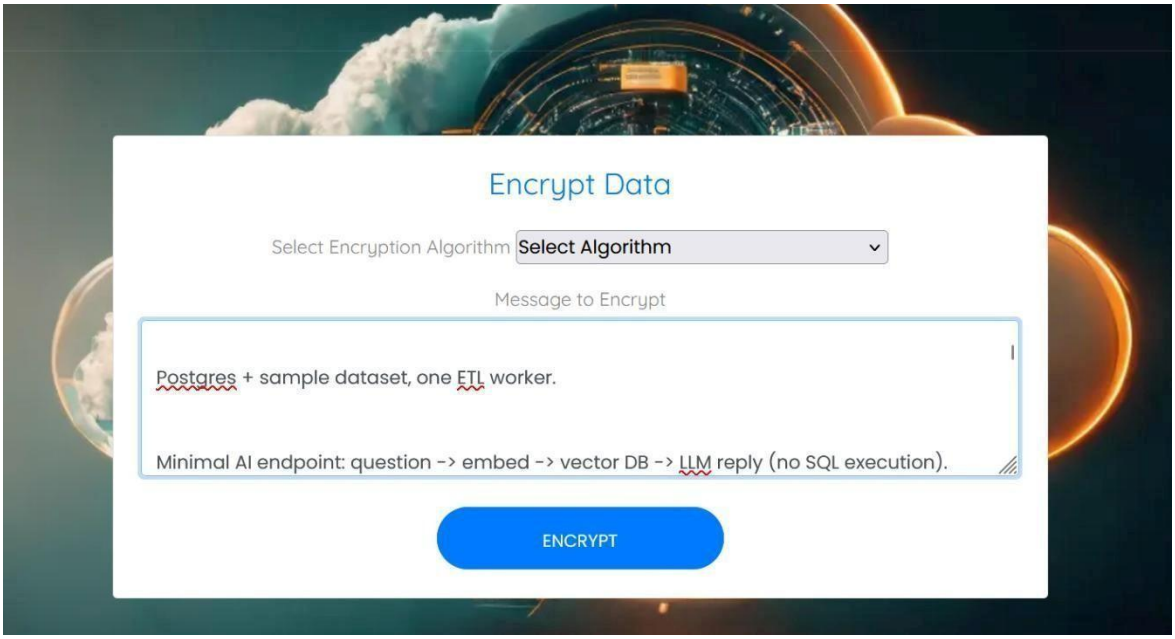


Discussion:

This module makes sure that encryption and decryption features are only accessible to authorized users. Unauthorized entry is prevented by the combination of backend session control (Django) and frontend validation (JavaScript).

10.2 Data Encryption Interface

The user goes to the Encrypt Data section after logging in. Here, the user uploads a file to be encrypted or enters a message. A dropdown list allows the user to choose between AES and ECC as the encryption algorithm. The system uses the selected algorithm to transform the plaintext into a ciphertext when the "Encrypt" button is pressed.

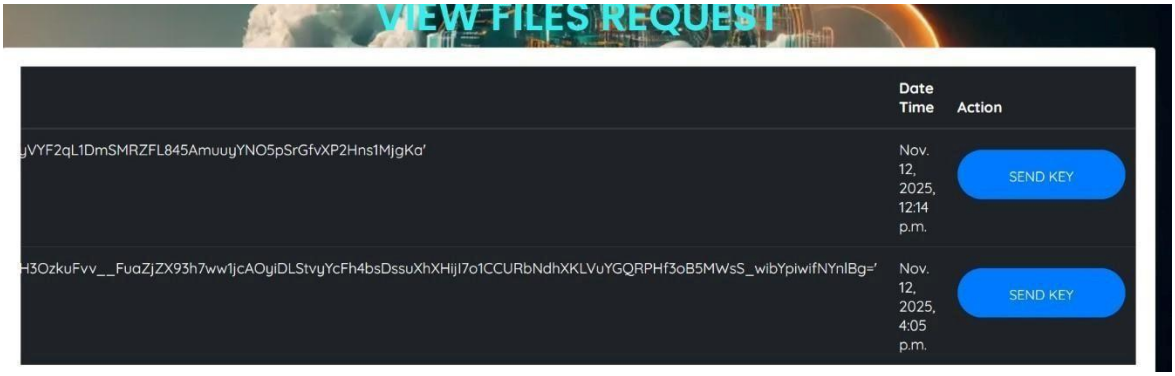


Discussion:

- Data confidentiality is accelerated by AES (Advanced Encryption Standard). To increase security during key sharing, ECC (Elliptic Curve Cryptography) incorporates asymmetric key exchange.
- To guarantee data integrity and traceability, the encrypted file is safely kept in the database and registered on the Ganache blockchain.

10.3 File Request and Key Management

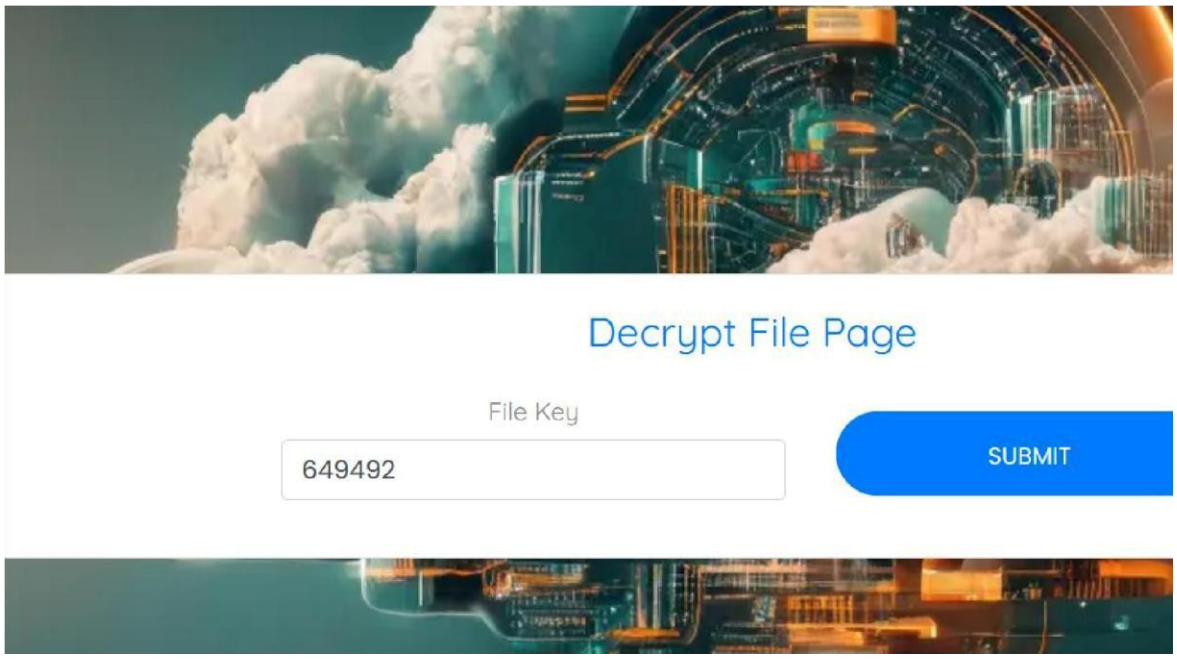
All encrypted files are shown on the File Request Page, along with information like the date, time, and hash of the encryption key. By selecting the "Send Key" button, which initiates an automated email with a special decryption code, the Data Owner can authorize a user's request for access to a file.



This method guarantees that decryption keys are never kept in plain form or made available to users directly. Man-in-the-middle attacks are prevented and confidentiality is strengthened by the ECC-based key exchange and email delivery system. A tamper-proof audit trail for key transfers is also provided by keeping the transaction record on the Ganache blockchain.

10.4 Decryption Using Unique Code

The user can unlock the encrypted data by entering the unique decryption code they received via email on the Decryption Page. Depending on the encryption algorithm that was previously used, the system decrypts the code using either AES or ECC after verifying it.

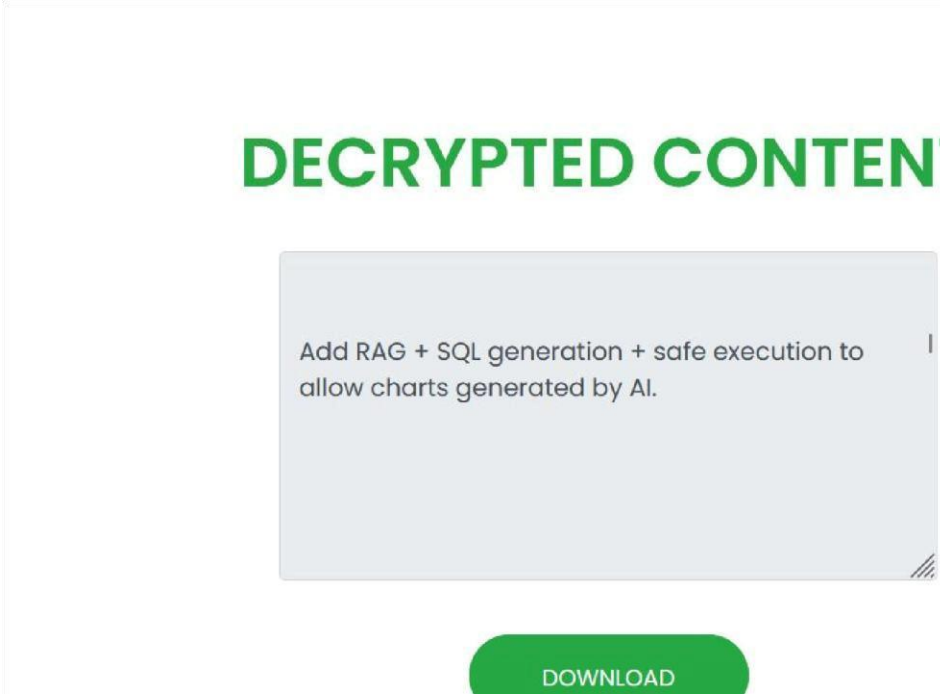


Discussion:

By adding an extra layer of authentication, this feature makes sure that even if encrypted files are intercepted, they cannot be decrypted without the right code. The use of email-based verification improves security and usability.

10.5 Decrypted Output Display

The system decrypts the data and shows the user the original message or file content after the verification is successful. The interface allows the user to safely view or download the decrypted content.

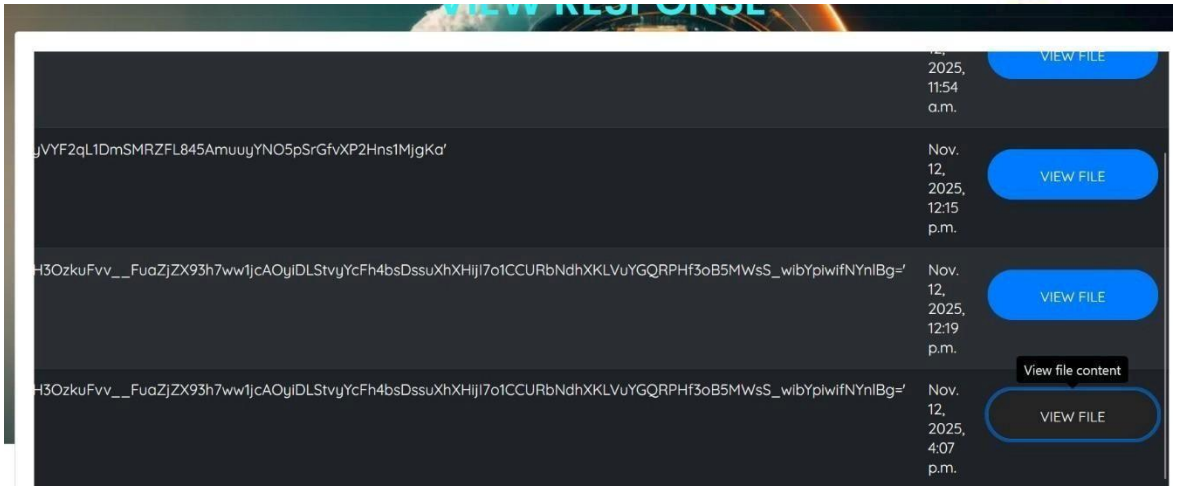


Discussion:

The encryption-decryption process' accuracy is verified by this module. The output verifies that the blockchain-based key management procedure operates dependably and that the AES algorithm perfectly restores the original plaintext.

10.6 View Files Interface

All of the encrypted and decrypted files that belong to the user who is currently logged in are listed on the View Files page. It contains information like the filename, encryption date and time, algorithm used, and request status.


VIEW RESPONSE
VIEW FILE
2025, 11:54 a.m.
yVYF2qL1DmSMRZFL845AmuuyYNO5pSrGfvXP2Hns1MjgKa'
Nov. 12, 2025, 12:15 p.m.
VIEW FILE
H3OzkuFvv__FuaZjZX93h7ww1jcAOyiDLStvyYcFh4bsDssuXhXHijI7o1CCURbNdhXKLVuYGQRPHf3oB5MWsS_wibYpiwifNYnlBg='
Nov. 12, 2025, 12:19 p.m.
VIEW FILE
View file content
H3OzkuFvv__FuaZjZX93h7ww1jcAOyiDLStvyYcFh4bsDssuXhXHijI7o1CCURbNdhXKLVuYGQRPHf3oB5MWsS_wibYpiwifNYnlBg='
Nov. 12, 2025, 4:07 p.m.
VIEW FILE

CHAPTER 11

Conclusion andFuture Work

Conclusion

The suggested system, Dynamic AES Encryption and Blockchain Key Management for Cloud Data Security, effectively manages data stored in the cloud in a transparent, safe, and effective manner. The system guarantees a multi-layered approach to data protection by combining Blockchain (Ganache) with AES (Advanced Encryption Standard) and ECC (Elliptic Curve Cryptography).

The project shows how ECC facilitates secure key exchange while AES offers quick and dependable symmetric encryption for data confidentiality. The Ganache blockchain's integration makes it possible to log important transactions indefinitely, guaranteeing the system's integrity and transparency.

Furthermore,

The process of distributing keys via email improves authentication by guaranteeing that only authenticated users is able to access and decrypt private documents. The system's Django-based web interface makes it

simple for users to sign up, log in, and

upload files, securely decrypt content, encrypt data, and request decryption keys. By means of experimentation, the system demonstrated efficacy in thwarting unwanted access, preserving maintaining confidentiality and guaranteeing complete security in cloud settings.

All things considered, this system offers a strong foundation for blockchain, key management, and data security. based traceability, fixing a number of flaws in conventional cloud storage systems.

Future Work

Even though the system significantly improves cloud data security, there is still room for development and expansion in subsequent studies and applications. Future improvements could include the following:

1. **Integration with Real Blockchain Networks:**

   Deploying the system on a public Ethereum or Hyperledger network rather than the local Ganache test network could offer decentralized worldwide verification and practical scalability.

2. **Performance Optimization:**

   By optimizing algorithm selection and database handling, future iterations can increase encryption speed and blockchain transaction time, particularly for large datasets or multimedia files.

3. **Multi-Factor Authentication (MFA):**

   The login and decryption procedures can be strengthened even further by adding biometric or OTP-based authentication layers.

4. **Machine Learning–Based Threat Detection:**

   Real-time detection of anomalies or possible security breaches could be facilitated by using AI models to analyze access patterns.

5. **Cloud Integration:**

   To ensure dynamic encryption during file uploads and downloads, the system can be expanded to function directly with cloud platforms like AWS S3, Google Cloud Storage, or Microsoft Azure.

6. **User Role Expansion:**

   To improve workflow control and auditing, future iterations may include various access levels, such as admin, auditor, and data analyst.

7. **Enhanced Key Rotation and Revocation:**

Long-term data protection would be enhanced by implementing blockchain-based key revocation procedures and automated key rotation policies.

CHAPTER 12

REFERENCES/BIBLIOGRAPHY

[1]. R. Anandkumar, K. Dinesh, A. J. Obaid, P. Malik, R. Sharma, A. Dumka, R. Singh, and S.

Khatak, ‘‘Securing e-health application of cloud computing using hyperchaotic image encryption framework,’’ Comput. Electr. Eng., vol. 100, May 2022, Art. no.
107860.

2 Z. Bashir, T. Rashid, and S. Zafar, ‘‘Hyperchaotic dynamical system based image encryption scheme with time-varying delays,’’ Pacific Sci. Rev. A, Natural Sci. Eng., vol. 18, no. 3, pp. 254– 260, Nov. 2016.

3 W. Y. Chang, H. Abu-Amara, and J. F. Sanford, Transforming Enterprise Cloud Services.

Berlin, Germany: Springer, 2010.

4 B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, and M. Ayaz, ‘‘A systematic literature review on cloud computing security: Threats andmitigation strategies,’’ IEEE Access, vol. 9, pp. 57792–57807, 2021.

5 N. M. Sultana and K. Srinivas, ‘‘Survey on centric data protection method for cloud storageapplication,’’ in Proc. Int. Conf. Comput. Intell. Comput. Appl. (ICCICA), Nov. 2021, pp. 1–8.

6 F. Thabit, O. Can, S. Alhomdy, G. H. Al-Gaphari, and S. Jagtap, ‘‘A novel effective

lightweight homomorphic cryptographic algorithm for data securityin cloud computing,’’ Int. J.

Intell. Netw., vol. 3, pp. 16–30, 2022.

7 D. C. Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, ‘‘Integration of blockchain and cloud of things: Architecture, applications and challenges,’’ IEEE Commun. Surveys Tuts., vol. 22, no. 4, pp. 2521–2549, 4th Quart., 2020.

8 S. N. G. Gourisetti, Ü. Cali, K.-K.-R. Choo, E. Escobar, C. Gorog, A. Lee, C. Lima, M. Mylrea, M. Pasetti, F. Rahimi, R. Reddi, and A. S. Sani, ‘‘Standardization of the distributed ledger technology cybersecurity stack for power and energy applications,’’ Sustain. Energy, Grids Netw., vol. 28, Dec. 2021, Art. no. 100553.